EVOLUTION COOKBOOK

WAFT | Recipes for Breeding Better Agents

Introduction

This cookbook contains practical recipes for evolving agents using WAFT. Each recipe addresses a specific evolutionary goal.

Recipes

Recipe 1: Basic Agent Evolution

Goal: Evolve a simple agent through 5 generations

Ingredients

- Base agent class
- Mutation configuration
- Fitness function (Scint Gym)
- 5+ generations of patience

Instructions

Python

```python
from waft import Laboratory, Agent, Evolution

# 1. Create laboratory
lab = Laboratory("basic_evolution")

# 2. Define base agent
class SimpleAgent(Agent):
    def process(self, input_text: str) -> str:
        return f"Processed: {input_text}"

# 3. Configure evolution
evo = Evolution(
    population_size=10,
    mutation_rate=0.1,
    generations=5,
)

# 4. Run evolution
results = evo.run(
    base_agent=SimpleAgent,
    fitness_fn=lambda a: a.scint_score(),
)

# 5. Get best agent
champion = results.best_agent()
```

Expected Results

- Generation 0: Baseline fitness 0.5
- Generation 5: Improved fitness 0.7-0.8
- Flight Recorder: 50+ events logged

Recipe 2: Prompt Mutation

Goal: Evolve agent prompts for better Scint handling

Ingredients

- Agent with system prompt
- Prompt mutation templates
- Semantic similarity scorer

Instructions

Python

```python
from waft.mutations import PromptMutator

# Define mutation strategies
mutator = PromptMutator(
    strategies=[
        "rephrase",      # Same meaning, different words
        "elaborate",     # Add detail
        "simplify",      # Remove complexity
        "specialize",    # Add domain focus
    ],
    temperature=0.7,
)

# Apply mutations
variants = mutator.generate_variants(
    base_prompt="You are a helpful assistant.",
    count=10,
)

# Evaluate each variant
for variant in variants:
    agent = Agent(system_prompt=variant)
    score = gym.evaluate(agent)
    flight_recorder.log(agent, score)
```

Pro Tips

- Start with simple prompts
- Evaluate on diverse Scint types
- Keep successful mutations for breeding

Recipe 3: Code Genome Evolution

Goal: Evolve agent source code directly

Ingredients

- Agent with mutable code regions
- AST-aware mutation engine
- Strong test coverage

Instructions

Python

```python
from waft.genome import CodeGenome, ASTMutator

# Mark mutable regions
class EvolvableAgent(Agent):
    # @mutable: This method can be evolved
    def analyze(self, data: dict) -> dict:
        # Base implementation
        return {"result": data.get("input", "")}

# Create genome
genome = CodeGenome.from_agent(EvolvableAgent)

# Configure mutations
mutator = ASTMutator(
    allowed_operations=[
        "add_conditional",
        "modify_expression",
        "add_error_handling",
        "optimize_loop",
    ],
)

# Evolve
for generation in range(10):
    variants = mutator.mutate(genome, count=5)
    scores = [gym.evaluate(v) for v in variants]
    genome = select_best(variants, scores)
```

Warning

Code mutations can break agents. Always validate syntax before evaluation. Use sandboxed execution.

Recipe 4: Multi-Agent Tournament

Goal: Evolve agents through competition

Ingredients

- Multiple agent lineages
- Tournament selection
- Shared fitness landscape

Instructions

Python

```python
from waft.tournament import Tournament

# Create competing lineages
lineages = [
    Lineage("conservative", mutation_rate=0.05),
    Lineage("aggressive", mutation_rate=0.2),
    Lineage("balanced", mutation_rate=0.1),
]

# Configure tournament
tournament = Tournament(
    lineages=lineages,
    rounds=20,
    selection="tournament",  # Top performers breed
    elimination=True,        # Bottom performers die
)

# Run tournament
results = tournament.run()

# Analyze
print(f"Winning lineage: {results.champion.lineage}")
print(f"Generations survived: {results.generations}")
print(f"Final fitness: {results.champion.fitness}")
```

Analysis

Different mutation rates suit different environments:

- Low mutation: Stable, slow improvement
- High mutation: Volatile, fast exploration
- Balanced: Good default choice

Recipe 5: God-Head Pursuit

Goal: Long-term evolution toward emergent intelligence

Ingredients

- Large population (100+)
- Many generations (1000+)
- Complex fitness landscape
- Patience and compute

Instructions

Python

```python
from waft.godhead import GodHeadPursuit

# Configure long-term evolution
pursuit = GodHeadPursuit(
    population_size=100,
    generations=1000,
    fitness_dimensions=[
        "scint_stability",
        "task_generalization",
        "self_modeling",
        "goal_generation",
    ],
    checkpoints_every=50,
)

# Define emergence criteria
pursuit.set_emergence_criteria({
    "self_reference": 0.8,  # Agent discusses own cognition
    "novel_goals": 0.7,     # Agent creates new objectives
    "meta_learning": 0.9,   # Agent improves its learning
})

# Run (this takes a while)
results = pursuit.run()

# Check for emergence
if results.emergence_detected:
    print("🎉 God-Head candidate found!")
    candidate = results.emerged_agents[0]
    save_for_study(candidate)
```

The Long Game

This is the ultimate goal of WAFT. We don't know what a God-Head looks like—that's why we need evolution to discover it.